

Implementation plan — every step, in full detail

This expands each stub in `src/steps/` into exact inputs, algorithms, formulas, data shapes, outputs, and acceptance checks. It is the "fill-in-the-blanks" companion: the docstrings say *what*; this says *exactly how*. Reference implementations for the statistical steps already exist in `src/pipeline.py` (the integrated donor-level script).

Conventions used throughout

- **Paths:** import `config` (`PAPER1`, `PAPER2_TRAIN`, `SIGNATURES`, `PC_GENES/PP_GENES` = the Paper 2 landmark baseline, `FIGURES`, `TABLES`).
 - **Matrix orientation:** `counts.mtx` is genes $\times$ cells (30117×69426), raw integer UMIs. Row $i \leftrightarrow$ `genes.txt[i]`; col $j \leftrightarrow$ `barcodes.txt[j]`.
 - **Donor** = `metadata_all_cells.csv["Patient.ID"]`. **Stage** = `cell_metadata.csv["stage"]`, ordered: Healthy control < NAFLD < NASH w/o cirrhosis < NASH with cirrhosis < end stage $\rightarrow$ ranks 0.4 (S2R).
 - **GOLDEN RULE:** unit of inference = **donor** (~ 47). Aggregate cells $\rightarrow$ one number per donor, then test on the donor values. Cell-level p-values are pseudoreplication.
 - **The A4 table** (output of Step 4, consumed by 5–8): columns `cell_id`, `donor`, `stage`, `zonation_coord`, `pc`, `pp`, `plasticity`, `zone_p0`, `zone_p1`, `zone_p2`, `entropy`.
-

Step 1 — Build the ruler (prep; DONE)

Inputs: Paper 2 landmark CSVs $\rightarrow$ `signatures/*_paper2_landmark.txt` (baseline, 20+20); `supplementary_table_8.xlsx` sheet Hepatocyte $\rightarrow$ `signatures/*_expanded.txt` (ranked, ~ 100); `combined_scRNAseq_atlas_M5M6M7M8.mat` $\rightarrow$ `paper2_train.npz`. **Training labels (the classifier's y):** zone label per Paper 2 nucleus. - *Ground-truth (target):* port `parse_snRNAseq_combined_atlas.m` — Paper 2 mapped its **spatial** (Visium) zonation onto its snRNA nuclei; use that zone index, binned to 3 classes. - *Fallback (current `paper2_train.npz`):* `zone_label = tercile( score(PC) - score(PP) )` computed on Paper 2's own hepatocytes (0 portal / 1 mid / 2 central). **Artefact A1.** Acceptance: 3 zone classes present with balanced-ish counts; PC genes rise / PP genes fall along the zone index (profile plot, `plot_a1_reference_profiles`).

Step 2 — Load & QC (`steps/step2_load_qc.py`; ref `pipeline.py:load`)

Inputs: `PAPER1/{counts.mtx, genes.txt, barcodes.txt, cell_metadata.csv, metadata_all_cells.csv}`. **Algorithm** 1. `M = scipy.io.mmread(PAPER1/"counts.mtx").tocsc()` $\rightarrow$ genes $\times$ cells sparse. 2. `genes = [l.strip() ...]`, `bars = [l.strip() ...]` (order = matrix rows/cols). 3. `meta = read_csv(cell_metadata).set_index("cell_id").reindex(bars)  $\rightarrow$  stage = meta["stage"]`. 4. `allm = read_csv(metadata_all_cells).set_index("cell_id")`; auto-detect donor column from `["Patient.ID", "patient", "donor", "orig.ident", "sample"]`; `donor = allm[col].reindex(bars)`. 5. `libsize = asarray(M.sum(0)).ravel()` (per-cell total UMIs). 6. **QC sanity only (do NOT re-filter):** confirm kept cells are hepatocytes — mean ALB expression high; print per-stage and per-donor counts. Optional sensitivity: flag the lowest-nCount_RNA decile for a robustness re-run, don't delete. **Normalization (used by scoring, applied per gene as needed — `zrows()`):** for a gene row x , $v = \log_{1p}(x / \text{libsize} * 1e4)$ (CP10k $\rightarrow$ log1p), then per-gene z : $z = (v -$

$\text{mean}(v) / \text{std}(v)$ (if $\text{std}=0$, $z=0$). **Output A2:** M, genes, bars, stage, donor, libsize + a per-stage/per-donor count table. **Acceptance:** 69,426 hepatocytes; 5 stages with counts 3750/7073/31903/3603/23097; ALB clearly expressed.

Step 3 — Harmonize genes (steps/step3_harmonize.py)

Inputs: A2 genes + signature gene sets (+ paper2_train.feats for the classifier). **Algorithm** 1. Reconcile symbols (and Ensembl IDs if present) across Paper 1, Paper 2, signatures; report mapping rate $| \text{shared} | / | \text{signature} |$. 2. Restrict signatures + classifier features to the shared set; if a signature drops below ~15 genes, widen via the expanded list. 3. Choose batch-robust features: per-gene z-score (above) and/or rank-transform, so the score measures portal-vs-central rather than snRNA-vs-Visium platform. **Output A3:** harmonization report (mapping rate, surviving signature sizes) + frozen gene sets. **Acceptance:** $\geq 80\%$ of signature genes survive the intersection.

Step 4 — Place every cell

4a Signature scoring (steps/step4_score.py; ref pipeline.py:score)

For a gene set S , the **background-subtracted score** (Scanpy `score_genes` style):

```
Y = z-scored log1p-CP10k expression (genes × cells)
score_S[c] = mean_{g in S} Y[g,c] - mean_{g in B} Y[g,c]
```

where B = a control set drawn to match the expression-bin distribution of S (subtracting B removes each cell's overall expression/depth baseline, so the score reflects S specifically). Then:

```
pc = score(PERICENTRAL); pp = score(PERIPORTAL)
zonation_coord = pc - pp # signed; >0 pericentral, <0 periportal
plasticity = score(PLASTICITY) # signatures/plasticity.txt:
KRT7, KRT19, SOX9, SOX4, KRT23, NCAM1
```

Default gene sets: `*_paper2_landmark.txt`. (Our simpler `pipeline.py:score` uses the mean of z-scored signature genes without an explicit control set; the `score_genes` background form above is the upgrade.)

4b Classifier + entropy (steps/step4b_classifier.py; ref classifier.py)

1. Train on `paper2_train.npz`: X (nuclei $\times$ 40 landmark feats, CP-normalized), $y = \text{zone_label} \in \{0,1,2\}$.
2. `Pipeline(StandardScaler(), LogisticRegression(multi_class="multinomial", C=..., max_iter=...))`. Prefer **calibrated** logistic (well-behaved probabilities) over random forest.
3. **Evaluate on a held-out Paper 2 split FIRST** (`train_test_split`, stratified by donor): accuracy, confusion matrix; must beat chance (1/3) clearly.
4. Apply to Paper 1 (same 40 feats, same normalization): $P = \text{clf.predict_proba}(X1) \rightarrow \text{zone_p0..2}$.
5. **Entropy** per cell: $H[c] = - \sum_k p_k \log p_k$ (k over the 3 zones; natural log; range $[0, \log 3]$). Low H = confident zone; high H = de-zonated/confused.

6. **4c agreement:** `spearman(zonation_coord, expected_zone)` on Paper 1 (esp. healthy) > ~0.5.

Output A4: merge `zone_p0..2`, entropy into the per-cell table.

Step 5 — Validate on healthy (`steps/step5_validate.py`; ref `pipeline.py:validate`) — GATE

Input: A4 restricted to Paper 1 healthy donors. **Algorithm:** for each validation marker, `rho = spearman(coord[h], expr_marker[h])`. - pericentral CYP2E1, CYP1A2, GLUL, PCK2 → expect `rho > 0`. - periportal ASS1, ALDOB, PCK1, HAL → expect `rho < 0`. - classifier entropy in healthy should be low (peaked predictions). **Output A5:** bar of `rho` per marker (`plot_a5_validation`) + the values. **Acceptance / GATE:** pericentral `rho>0`, periportal `rho<0`. **Do not start Step 6 until this passes.** If it fails: widen to expanded signatures, use rank features, revisit normalization.

Step 5b — Is the ruler still a ruler? (`steps/step5b_ruler_diagnostics.py`)

Per stage `s`, on cells of that stage: 1. **Internal coherence:** mean pairwise `corr` among pericentral genes (and among periportal). Healthy = high; if it crashes, the module is dissolving (not just de-zonating). 2. **Cross-program anti-correlation:** `corr(pc, pp)` across cells. Healthy ≈ strongly negative; weakening toward 0 = true de-zonation (robust to global shifts). 3. **Split-half reproducibility:** randomly split each signature in two, build two coordinates, `corr` them. Drop = signature no longer self-consistent. 4. **Program-off vs restriction-lost:** track each module's mean magnitude (is it switched off?) vs the rate of cells co-expressing both ends (is positional restriction lost?). **Output A5b:** per-stage curves (`plot_a5b_ruler`). Tells you *which kind* of breakdown each stage is.

Step 6 — Collapse curve, H1 (`steps/step6_collapse.py`; ref `pipeline.py:collapse`)

Per-donor metrics (donors with ≥30 cells), one row per donor:

```
spread[d]    = std( coord[donor==d] )           # narrows with disease
anticorr[d]   = spearman( pc[donor==d], pp[donor==d] ) # rises toward 0
entropy[d]    = mean( H[donor==d] )             # rises (if classifier run)
stage_rank[d] = S2R[ mode(stage[donor==d]) ]
```

Ordered-trend test on the ~47 donor values, per metric: - **Spearman:** `rho`, `p` = `spearmanr(stage_rank, metric)`; `rho = 1 - 6Σd²/(n(n²-1))` (no ties). - **Jonckheere-Terpstra** (ordered alternative across the 5 stage-groups): `J = Σ_{a<b} U(group_a, group_b)` where `U` counts pairs with `x_b > x_a`; standardize with the null mean $\mu_J = (N^2 - \sum n_s^2)/4$ and variance σ_J^2 (standard JT formula) → `z`, `p`. **Donor bootstrap CI:** resample donors with replacement `B=2000×`, recompute `rho`, take percentiles 2.5/97.5. **Negative control:** permute `stage_rank` across donors `B=2000×`, recompute `|rho|` → permutation `p = (#{null ≥ observed}+1)/(B+1)`. **Output A6:** `collapse_per_donor.csv` + `plot_a6_collapse` (donor points + per-stage mean) + per-metric `rho`, 95% CI, `p`, `perm_p`. **Acceptance:** `spread↓`, `anticorr→0`, (`entropy↑`); CI excludes 0; `perm_p` small.

Step 7 — Zone-resolved DE, H2 (steps/step7_de.py; ref pipeline.py:de)

Bin cells into zones by coordinate terciles: `terc = quantile(coord,[1/3,2/3])`, `zone = digitize(coord, terc) → 0 portal / 1 mid / 2 central`. **Pseudobulk** (the donor-level move): for each zone $z \in \{\text{portal}, \text{central}\}$, for each donor d with ≥ 20 cells in that zone, build one profile:

```
cells = (zone==z) & (donor==d)
cp     = per-cell CP-normalized counts (counts / colsum)
PB[d,:] = log1p( mean over those cells of cp * 1e4 )      # donor × gene
ranks[d]= stage_rank[d]
```

Test per gene (detected in $\geq 10\%$ of the zone's donors): - *MVP*: `rho, p = spearman(PB[:,g], ranks)` across donors. - *Cleaner H2 (the real test)*: OLS interaction on the pseudobulk `expr_g ~ coord + stage + coord:stage`; the **coord:stage coefficient** answers "does this gene's zonal slope weaken with stage?" — i.e. lost zonation, not just lost level. **Multiple testing**: `q = BH(p)` — sort `p` ascending; threshold $p(k) \leq (k/m) \cdot \alpha$; largest such k ; all below are significant; `q-value` = monotone-adjusted `p`. Report at $q < 0.05$. **Circularity guards (mandatory)**: flag `is_signature` genes; report significant counts BOTH including and EXCLUDING them; "driver" claims use non-signature genes; optionally a held-out gene split (define coordinate on half the zonation genes, test the other half). **Output A7**: `de_portal.csv`, `de_central.csv` (gene, effect, `p`, `q`, `is_signature`) + volcano (`plot_a7_volcano`). **Acceptance**: significant set survives excluding signature genes.

Step 8 — De-zonation ↔ plasticity, H3 (steps/step8_plasticity.py; ref pipeline.py:plasticity)

De-zonation proxy per cell: distance of `coord` from the committed ends, e.g. `dez = - |(coord - median(coord)) / std(coord)|` (near the middle = more de-zonated), or high entropy. **Confound**: both `dez` and `plasticity` rise with stage, so a POOLED correlation is fake-positive. Test **within donor**, then aggregate: - per donor d (≥ 30 cells): `r[d] = spearman(dez[donor==d], plasticity[donor==d])`; report `mean(r)` and % donors with $r > 0$. - **Regression** (the formal version): OLS `plasticity ~ dez + C(stage) + C(donor)`; the `dez` coefficient is the association *after removing* stage and donor; report its sign and `p`. - Optional two-group view: `MannWhitneyU(plasticity[de-zonated], plasticity[zonated])` within stage, with rank-biserial effect size. **Output A8**: stage-stratified scatter (`plot_a8_plasticity`) + within-donor stat + `dez` coefficient. **Acceptance**: within-donor (not pooled) effect reported with sign + `p`, honestly (positive or null).

Step 9 — Bonus: collapse vs inherited risk (steps/step9_bonus_enrichment.py)

Inputs: Step-7 hit list ($q < 0.05$, non-signature) = `hits`; Paper 3 risk-variant target genes = `risk`; background = genes **testable** in Step 7 (detected + tested), ideally expression-matched. **Test**: `k=|hitsnrisk|`, `n=|hits|`, `K=|backgroundnrisk|`, `N=|background|`; `p = hypergeom.sf(k-1, N, K, n)` (scipy); `fold = (k/n)/(K/N)`; BH across the few tests. **Output A9**: fold + CI + `q`. **Acceptance**: background well-defined; result stated with caveats (associational). Strictly optional.

Plotting machinery (`src/plotting/artefacts.py`)

fn	input	draws
<code>plot_a1_reference_profiles(profiles_df)</code>	gene × zone means	PC rising / PP falling along zone
<code>plot_a4_coordinate(coord_df)</code>	A4 table	coord histogram + 2 marker-vs-coord scatters
<code>plot_a5_validation(coord_df, markers)</code>	A4 healthy	bar of spearman(coord, marker), colored by expected sign
<code>plot_a5b_ruler(diag_df)</code>	per-stage diagnostics	coherence / anti-corr / split-half curves
<code>plot_a6_collapse(per_donor_df)</code>	<code>collapse_per_donor.csv</code>	donor points + per-stage mean (spread, anti-corr)
<code>plot_a7_volcano(de_df)</code>	<code>de_*.csv</code>	effect vs $-\log_{10} q$, signature genes flagged
<code>plot_a8_plasticity(coord_df)</code>	A4	stage-stratified dez-vs-plasticity scatter

Run order

prep (Step 1) → `run_p2_validation.py` (method positive control) → `run_all.py` (Steps 2–8) → Step 9 optional. All outputs land in `results/{tables,figures}/`. `run_all.py` reproduces every artefact from `data/processed/` on a clean checkout.